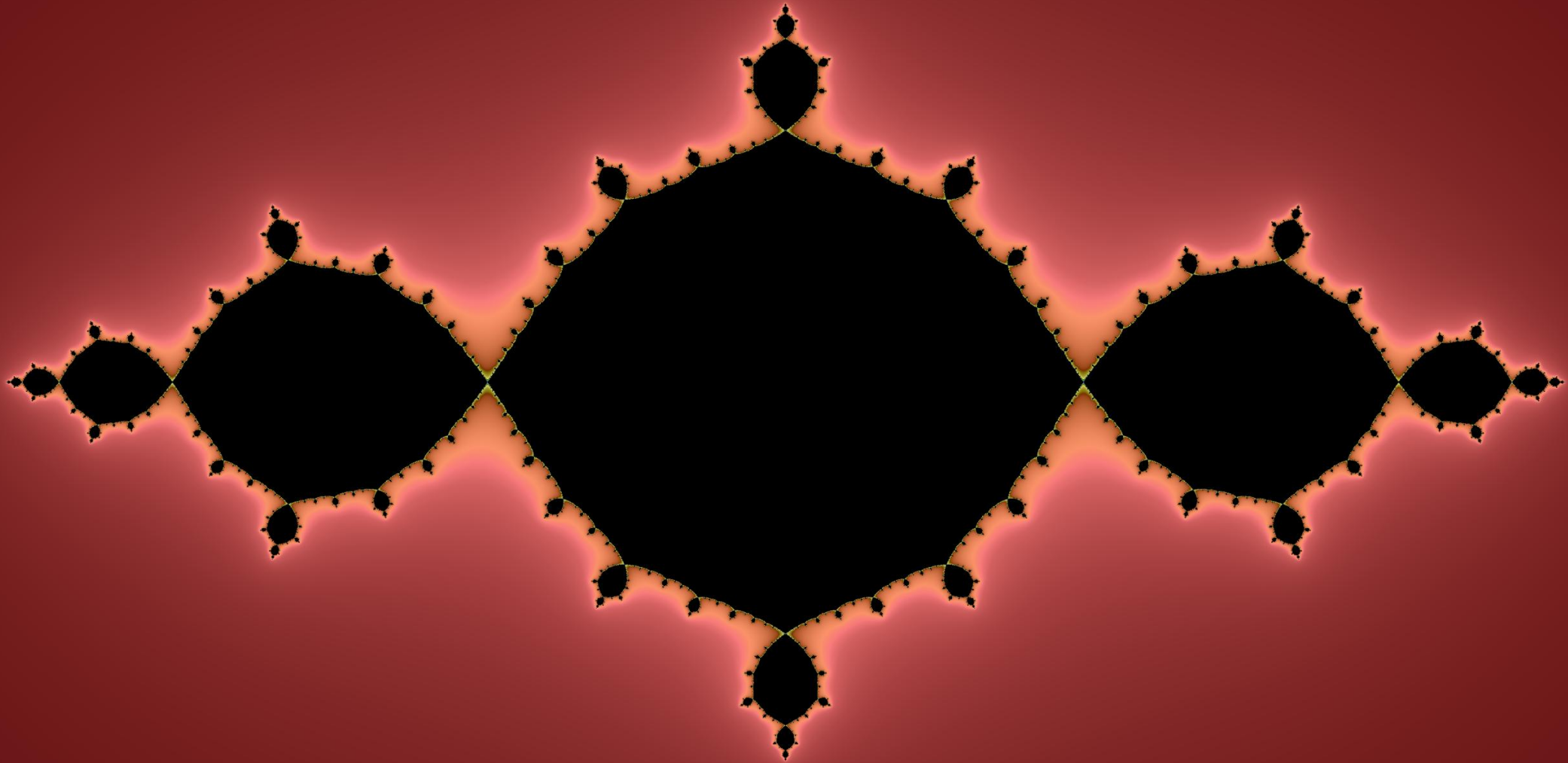


Fractal Software Design – Software Portability and Self Similarity (HAUD)

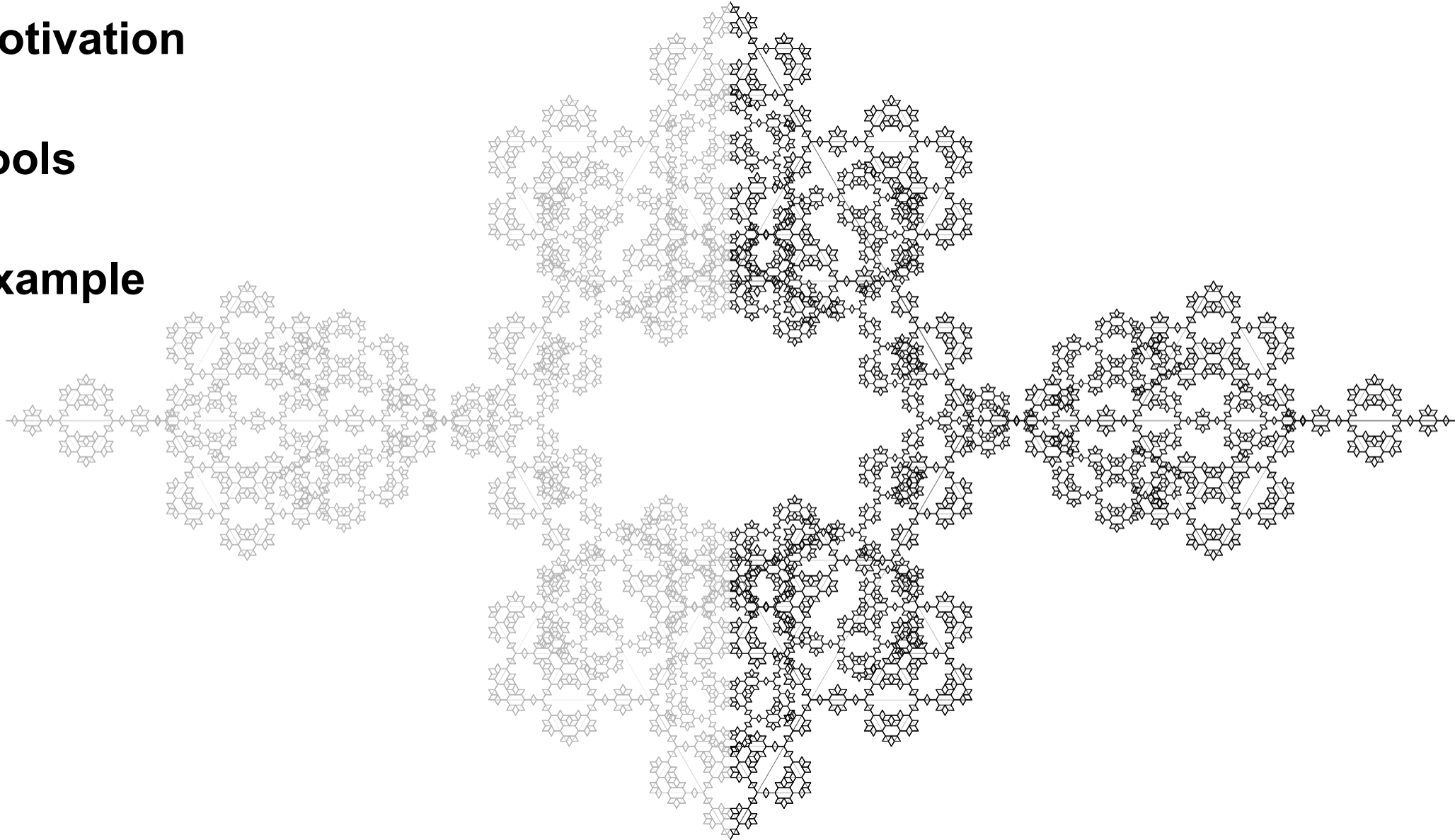
BECKHOFF



1. Motivation

2. Tools

3. Example



1. Motivation

- **Software portability**
 - Top level business logic shall not suffer from changes in lower abstraction levels.
- **Rigid hardware abstraction**
 - Unified software mechanics to shield business logic from hardware variance.
- **No God Objects**
 - Edge cases shall not disrupt abstraction.
- **Same cycle responsiveness**
 - Eliminate traditional, monolithic state machine design flaws.

1. Motivation

- **Software portability**
 - **Separation of business logic and motion logic enables portability.**
- **Changes in motion layer:**
 - Adding new features does not break the code.
 - Swapping a hardware controller does not break your business logic.
 - Introducing a new motion kernel does not break the code.

1. Motivation

- **Hardware Abstraction Layer**
 - **Strict interfaces enforce behavioural contract.**

Rigid = Enforced Contract = Safety

- **Rigid Interfaces ensure:**
 - Business logic never couples to hardware.
 - Hardware changes do not affect business logic.
 - Different hardware can be swapped transparently.

1. Motivation

- **NO God Objects**
 - **Work local, ship global.**
 - **Composition over inheritance.**
- **Composition ensures:**
 - Specialized logic can be reused.
 - Scales easily with future development.
- **Know Thyself:**
 - Know your job and do it well

1. Motivation

- **Same Cycle Responsiveness and Cycle Optimal Programming**
 - **Break the cycle.**
 - **Find a reusable structure to break safely.**
- **Logic progression:**
 - Do not waste time to switch decision trees.
 - Ensure observability.
 - Keep decisions simple

1. Motivation

– **WHY?**

- **Moving at 10 m/s requires cycle optimal programming.**
- **Synchronizing place and time with software that is fast and safe.**
- **The machine must observe itself and tell the user about it.**
- **Communication agnosticism is a requirement for distributed motion applications.**
- **Top application layer must be oblivious to the hardware executing the commands.**

2. Tools

▪ SOLID DESIGN

- **S** — Single Responsibility Principle
 - a class should have one, and only one, reason to change.
- **O** — Open/Closed Principle
 - software entities should be open for extension but closed for modification.
- **L** — Liskov Substitution Principle
 - Objects of a superclass shall be replaceable with objects of its subclasses without breaking the application.
- **I** — Interface Segregation Principle
 - users should not be forced to depend upon interfaces that they do not use.
 - a software entity only sees the methods it actually needs to do the work.
- **D** — Dependency Inversion Principle
 - Depend upon abstractions, not concretions.

2. Tools

▪ **ACID**-inspired principles for cyclic RTS

Borrowed from database transaction properties and adapted to deterministic control logic.

- **A** — Atomicity
 - Within a cyclic real-time task, a bounded local logic step can be treated as one deterministic, observable operation.
- **C** — Consistency
 - A transaction must bring the system from one valid state to another valid state.
 - The required handshake sequences perform a consistent execution. The behaviour is fully testable.
- **I** — Isolation
 - Concurrent transactions should not interfere with each other.
 - The architecture is isolated by design.
- **D** — Durability
 - Once a transaction has been committed, it will remain so, even in the event of power loss, crashes, or errors.
 - The loss of state or information must lead to a coordinated recovery.

2. Tools

▪ Clarifications

- The **SOLID** tools are directly applicable in TwinCAT OOP.
- The decision how deep inheritance runs is influenced by the task at hand.
- In my world, a flat inheritance tree has certain advantages regarding variations and the taming of complexity.
- Since there is no “one size fits all”
 - → the depth of inheritance is up for consideration
- Base classes convey the intent of custom extensions that solve an adjacent problem or substitute an existing implementation without changing the code.
- Interfaces are used as horizontal connector for classes that might share common properties.

2. Tools

▪ Clarifications

- The **ACID** tools require some explanations.
 - Transactional and relational principles share properties that are also applicable to PLC programming.
 - **Atomicity (Logic)**: The cyclic real-time task provides a deterministic execution boundary for bounded local logic. Within that boundary, internal decisions can be executed as one observable logical step.
 - We now only have to compress the traditional state machine changes (that consume a PLC cycle) into one observable sequence that executes without cycle change. By employing such sequential state execution, the logic needs only to wait for feedback when it is required.
 - Further is it helpful to eradicate the looping of arrays and classic buffers
 - Here the linked list comes in with just the right properties.

2. Tools

▪ Clarifications

- The **ACID** tools require some explanations.
 - A lot of similar problems are coded in vastly different ways, mostly caused by crunching time during FAE.
 - **Consistency (architecture)**: The facade pattern as helper
 - If everything works the same, reading the code is made easier.
 - Transactional handshakes have two trade offs
 - Easy to read
 - Easy to maintain

2. Tools

▪ Clarifications

- The **ACID** tools require some explanations.
- Shielding tested software against hacking is paramount.
- **Isolation (architecture)**: Again; the facade pattern as helper, and the horizontal Interface pointer as guard.
- If everything works an addition or deletion of functionality somewhere else must not break the architecture.
- Isolation by design is the best shield, if you require semaphores in a cyclic rts system you introduce an additional point of support once someone else touches your code.

2. Tools

▪ Clarifications

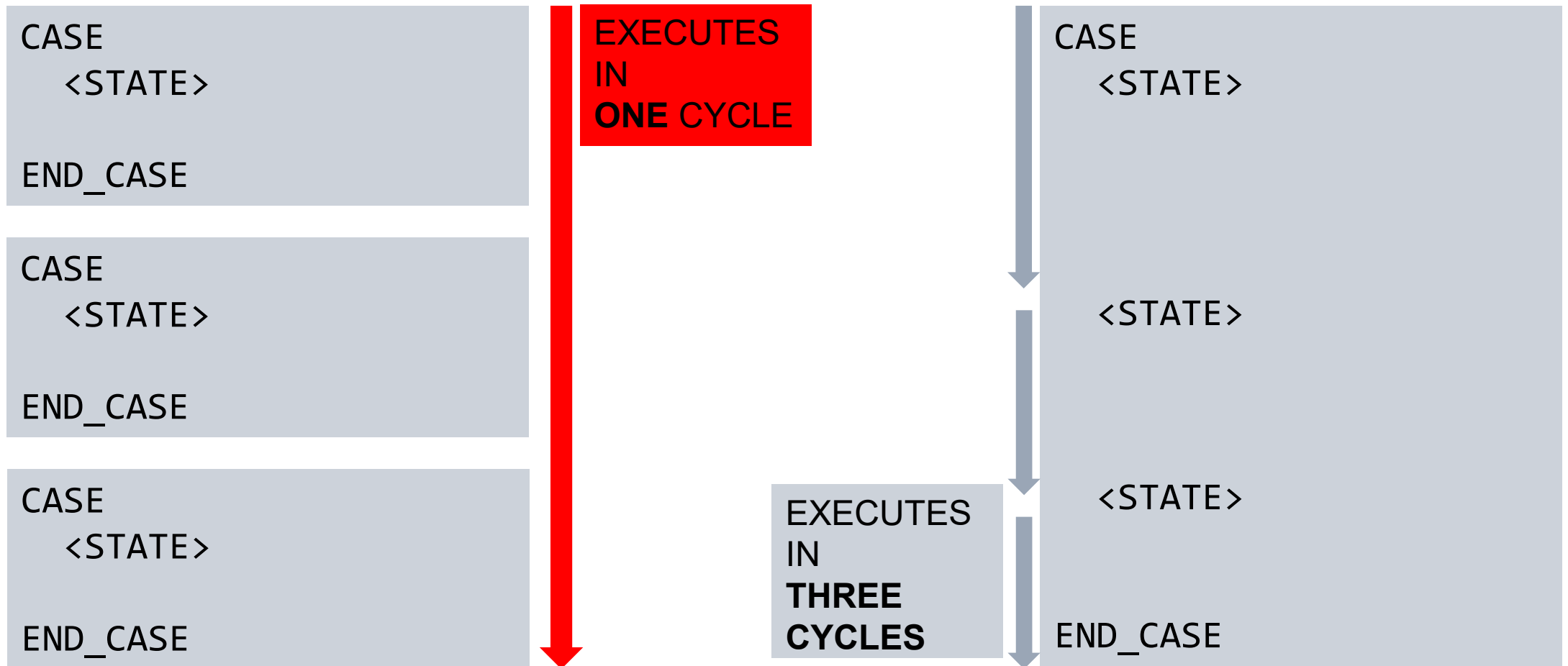
- The **ACID** tools require some explanations.
 - Saving data is one thing, saving machine state another
 - **Durability (design & physical constraint)**: On the topic of a machine, it is vital that machine parts are not colliding in the physical world.
 - Fetch the data you need for recovery in time.
 - Use UPS for your IPC → save persistent data
 - Use the request for interruption ahead of any physical reaction from the machine to save the data
 - Design a recovery strategy for the machine
 - A generic solution is either not working or complex as hell.

3. Examples

- **Break the cycle between states with E_PROGRESS.**
 - Core feature of the required Same Cycle Responsiveness.
 - Ensure logical atomicity for bounded local state progression in the cyclic RTS.
 - Logical atomicity applies to local decisions and coherent command publication. Physical effects are asynchronous and cannot generally be rolled back.
- Ensure machine readability without having to construct machine semantics on top.
- Ensure human readability without translation of numbers into text.
- Time Optimal Programming while keeping source code readable.

3. Examples

- Break the cycle between states with E_PROGRESS.



3. Examples

- Break the cycle between states with E_PROGRESS.

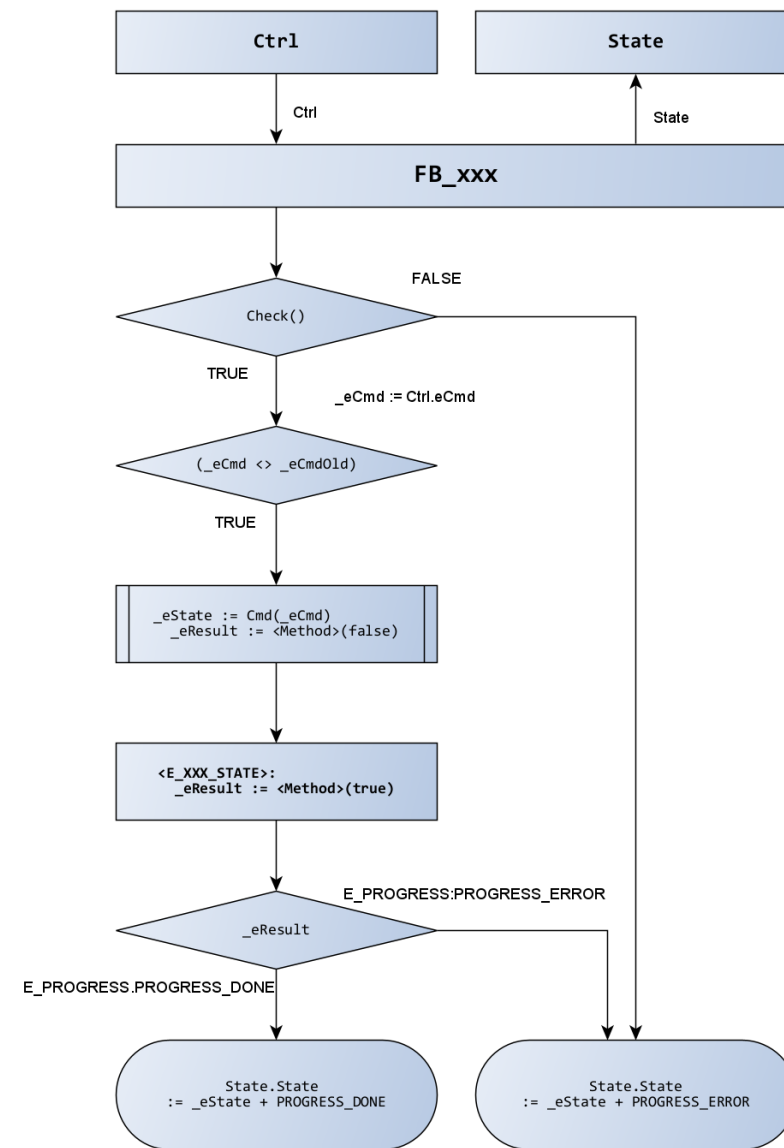
```
{attribute 'to_string'}
TYPE E_PROGRESS :
(
  // progress has 2 use cases
  // 1. as offset to cyclic interface's state
  //    for the requested command/function
  //    e.g. State := <Enum equivalent to eCmd> + E_PROGRESS
  //
  // 2. as state feedback (result) from (any) method
  PROGRESS_INVALID,
  PROGRESS_NOT_EXIST      :=  100,
  PROGRESS_INIT           := 1000,
  PROGRESS_BUSY           := 2000,
  PROGRESS_PREPARE        := 3000,
  PROGRESS_STARTUP        := 4000,
  PROGRESS_CHECK          := 5000,
  PROGRESS_OCCUPIED       := 6000,
  PROGRESS_WORKING        := 7000,
  PROGRESS_STILL_WORKING  := 8000,
  PROGRESS_ERROR          := 9000,
  PROGRESS_DONE           := 10000
) UINT;
END_TYPE
```



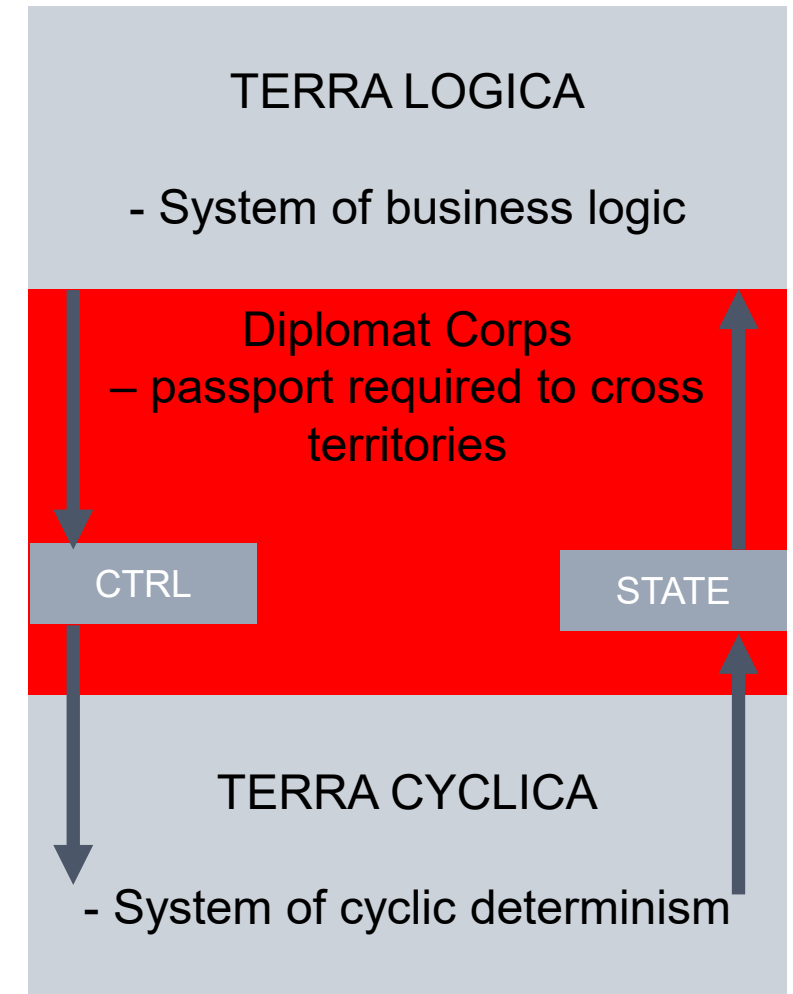
Shows
progress

3. Examples

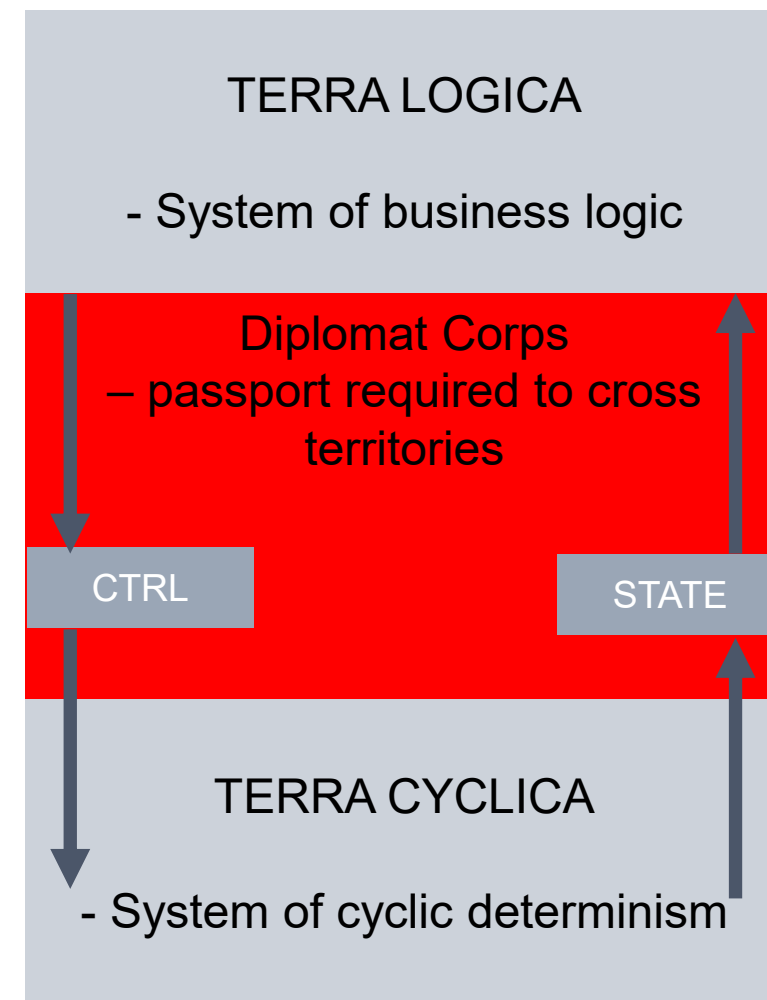
- **The Facade:**
 - State / Ctrl structures
 - Establishes unified access
 - Commands can simply be 'dropped' into Ctrl datafield
 - Enables asynchronous communication
 - Enables cyclic communication mapped onto any cyclic fieldbus
 - State is updated by own cyclic call
- State feedback for cyclic class wrappers
 - **Always** combined with E_PROGRESS
 - You can filter your response by a simple modulo division



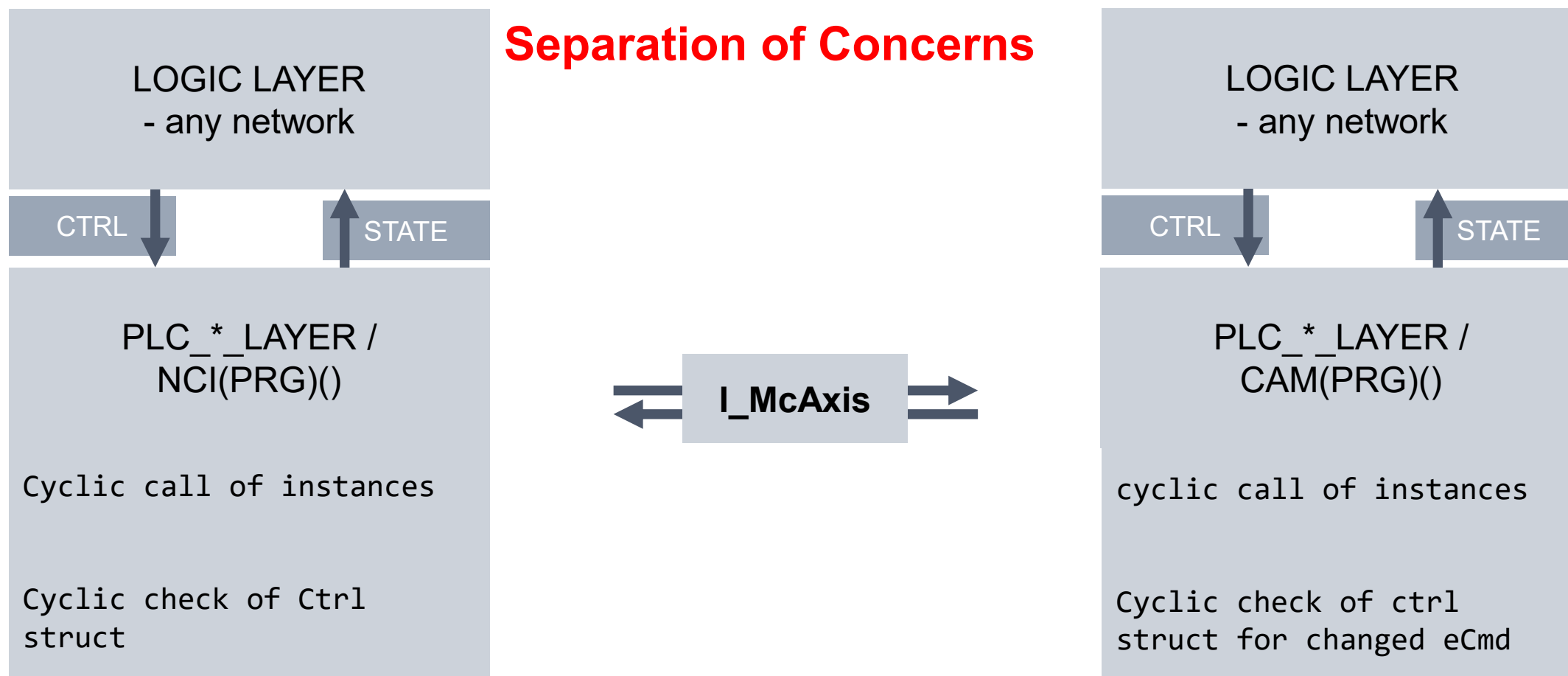
- **Diplomats and Territories - a tale of two systems**
- This world view depends on discipline not compiler restraints.
- The architecture operates on the principle of strict namespace encapsulation.
- The core TcGVL (Global Variable List) files are treated as sovereign territory.
- No external logic is permitted to read or write to this territory directly.



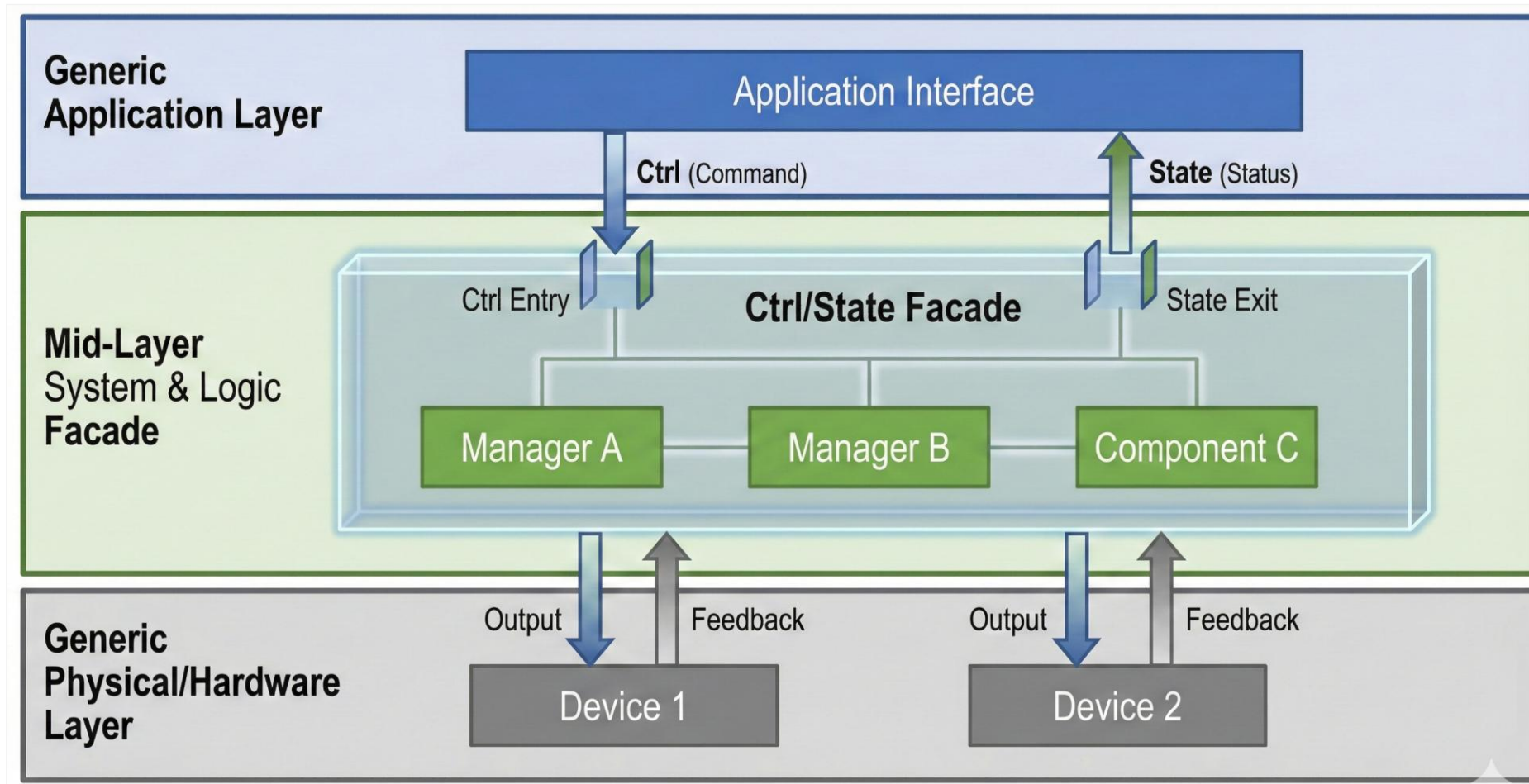
- **Diplomats and Territories - a tale of two systems**
- Instead, communication requires a "diplomat"—a strictly defined Facade or Interface.
- Any cyclic application may be connected via **MAPPING** procedures (fieldbus, EAP).
- Any non-cyclic application may be connected via symbolic access.
- Hybrid access possible
 - A middleware layer to organize access (switching between ADS and cyclic mapping) keeps the base layer unchanged



Software Design Example:



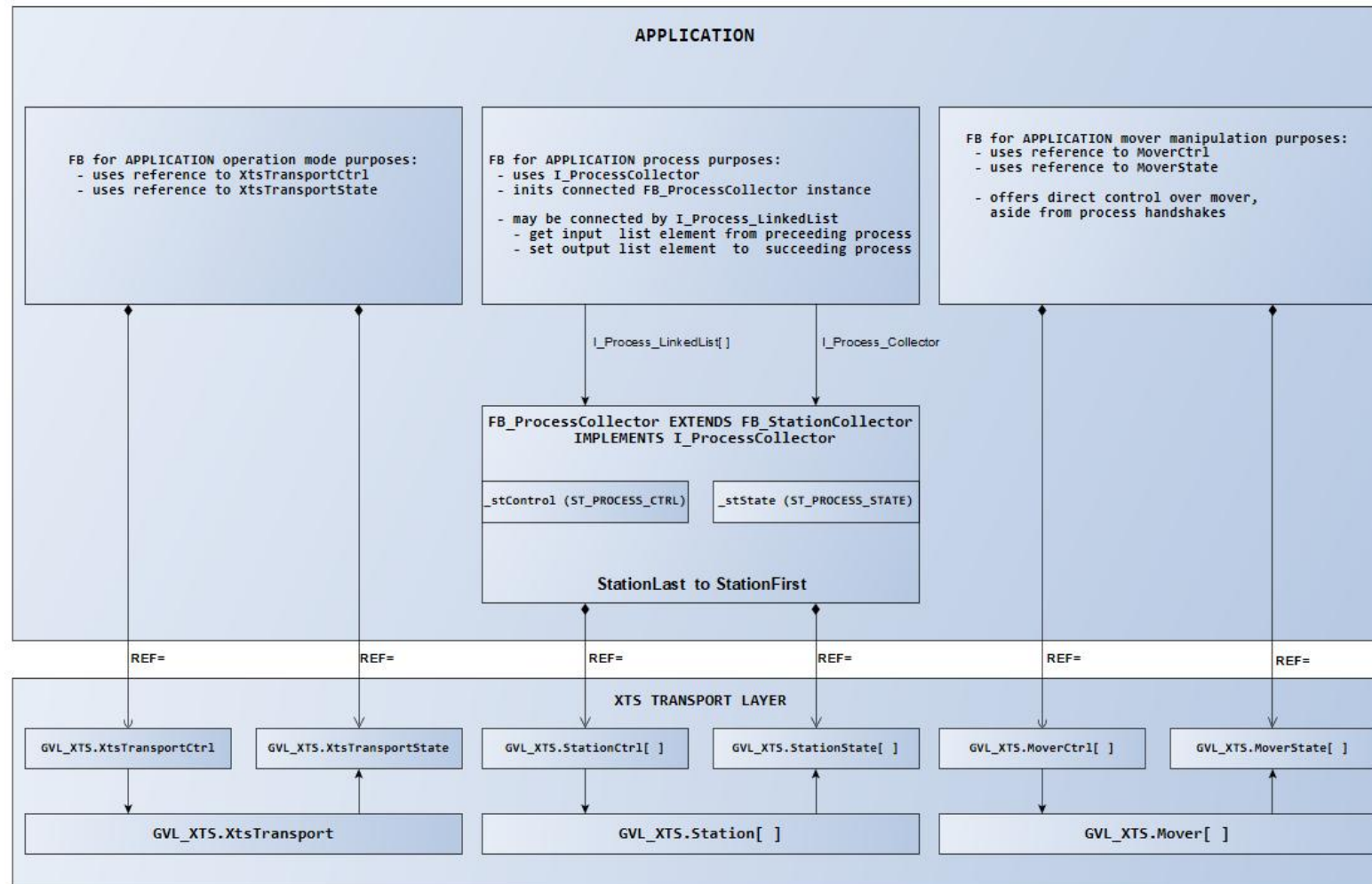
▪ Software Design Example: **The Facades**



■ Software Design Example:

- Build applications layer by layer
- The transport logic lives in instances

The Facades



▪ Software Design Example: **The Facades**

- **MAIN(XtsTransport)**
 - Read this file in code please, it shows everything

```
135 // cyclic call
136 GVL_XTS.XtsTransport();
137
138 // main control
139 GVL_XTS.XtsTransport.Ctrl      REF= GVL_XTS.XtsTransportCtrl; // main control
140 GVL_XTS.XtsTransport.State     REF= GVL_XTS.XtsTransportState; // main state
141
142 // member controls
143 GVL_XTS.XtsTransport.XpuCtrl   REF= GVL_XTS.XpuCtrl;
144 GVL_XTS.XtsTransport.XpuState  REF= GVL_XTS.XpuState;
145 GVL_XTS.XtsTransport.XpuInfo   REF= GVL_XTS.XpuInfo;
146
147 GVL_XTS.XtsTransport.GroupItf  := GVL_XTS.CaGroupItf;
148 GVL_XTS.XtsTransport.GroupInfo REF= GVL_XTS.CaGroupInfo;
149
150 GVL_XTS.XtsTransport.MoverItf  REF= GVL_XTS.MoverItf;
151 GVL_XTS.XtsTransport.MoverInfo REF= GVL_XTS.MoverInfo;
152 GVL_XTS.XtsTransport.MoverLastPosition REF= GVL_XTS.LastPosition;
153 GVL_XTS.XtsTransport.MoverLastGap REF= GVL_XTS.LastGap;
154
155 GVL_XTS.XtsTransport.StationStartIndex := GVL_XTS.StationStartIndex;
156 GVL_XTS.XtsTransport.StationStart     REF= GVL_XTS.StationStart; // start position for E_XTS_TRANSPORT_CTRL.CMD_TRANSPORT_START
157 GVL_XTS.XtsTransport.StationCtrlItf   REF= GVL_XTS.StationCtrlItf; // station interfaces are required for accessing station methods
158 GVL_XTS.XtsTransport.StationListsItf  REF= GVL_XTS.StationListItf; // list interface is required for access to linked list methods
159 GVL_XTS.XtsTransport.StationControl   REF= GVL_XTS.StationCtrl;
160 GVL_XTS.XtsTransport.StationState     REF= GVL_XTS.StationState;
```

Software Design Example:

GVL_APPLICATION (ExternControl)

- Read this file in code please, it explains everything.

The Facades

```

GVL_APPLICATION  x MAIN_APP
1  VAR_GLOBAL CONSTANT
2      MAX_PROCESS      : E_INSTANCE := E_INSTANCE.SENDER_BUFFER_INFEED;
3  END_VAR
4  {attribute 'qualified_only'}
5  VAR_GLOBAL
6      //-----
7      // Application
8      //
9      // this is the part you will have to change and adapt into your programming
10     //-----
11     //-----
12     // fb_Application:
13     // - procedures for getting fb_Instance classes and XTS_TRANSPORT_LAYER to work
14     //
15     // - Cmd/state behaviour similar to sw mechanics in fb_TransportUnit
16     // - E_INSTANCE_CMD / E_INSTANCE_STATE + E_PROGRESS
17     //   INIT          /   INIT          : initialize processes and XtsStations
18     //   DISABLE       /   DISABLED      : RemoveALLAxis from CA Group, disable CA Group, disable all movers
19     //   ENABLE        /   ENABLED       : AddALLAxis to CA Group, Enable CA Group, Enable all movers
20     //   WORK          /   WORKING       : CMD_TRANSPORT_START, enable handshakes, start writing list entries to BUFFER_INFEED
21     //   PROCEED       /   PROCEEDING    : CMD_TRANSPORT_RESTART, enable handshakes, continue writing list entries to BUFFER_INFEED
22     //   FINISH        /   FINISH        : stop writing list entries to BUFFER_INFEED, handshake processes until outfeeds are empty
23     //   CLEAR         /   CLEARED       : clear all instances of fb_Instance, clear all XtsStations
24     //-----
25     ApplicationMain    : fb_Application;
26     AppControl         : ST_APP_CTRL;
27     AppState          : ST_APP_STATE;
28
29     //-----
30     // fb_Instance control and state
31     // - structs are used in fb_Application as reference
32     //-----
33     InstanceCtrl       : ARRAY[1..MAX_PROCESS] OF ST_INSTANCE_CTRL;
34     InstanceState      : ARRAY[1..MAX_PROCESS] OF ST_INSTANCE_STATE;
35
36     TicketListControl  : ST_LIST_CTRL;
37     TicketListState    : ST_LIST_STATE;
38     TicketListDataInput : ST_LIST_NODE_DATA; // datafield for you to write(add) your data
39     TicketListDataOutput : ST_LIST_NODE_DATA; // datafield for you to read(get) your data
40 END_VAR
    
```

▪ Software Design Example: **The Facades**

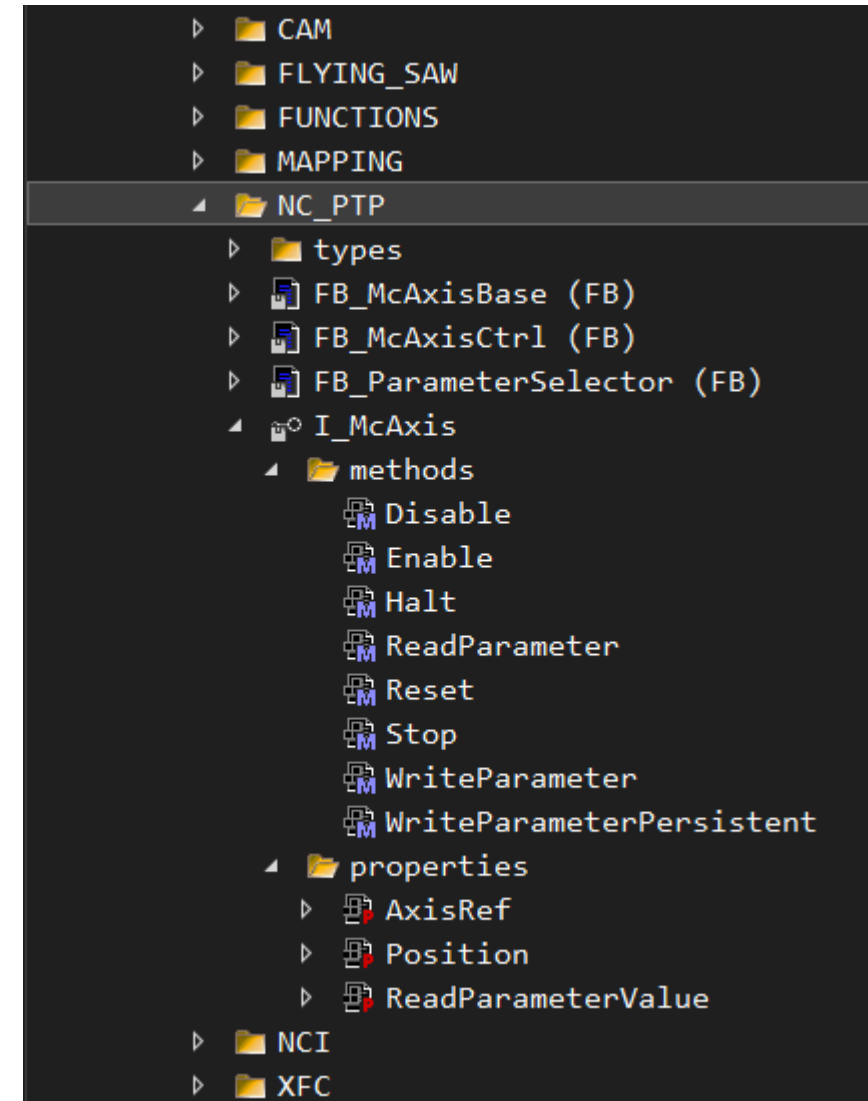
- **GVL_XTS(XtsTransport)**
 - Read this file in code please, it explains everything.

```
26
27 // ctrl / state datafields for extern control
28 StationCtrl      : ARRAY[1..MAX_STATION] OF ST_STATION_CTRL; // control for extern to write, you can do whatever you want as long as you do as I say
29 StationState     : ARRAY[1..MAX_STATION] OF ST_STATION_STATE; // state for extern to read, this is what I tell you to do
30
31 // station parameter
32 StationParameter : ARRAY[1..MAX_STATION] OF ST_STATION_PARAMETER; // wherever, whenever you want. Build it and they will come...
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49 XtsTransportCtrl : ST_XTS_TRANSPORT_CTRL; // control of XTS transport for extern to write
50 XtsTransportState : ST_XTS_TRANSPORT_STATE; // state of XTS transport for extern to read
51
52
53
54
55
56
57
58
59
60
61
62 //-----
63 // MC2 encapsulation, CA movement
64 Mover      : ARRAY[1..MAX_MOVER] OF fb_MoverCtrl; // MC and CA methods for mover; cyclic check for individual command on mover
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86 MoverCtrl      : ARRAY[1..MAX_MOVER] OF ST_MOVER_CTRL; // control of individual mover for extern to write
87 MoverState     : ARRAY[1..MAX_MOVER] OF ST_MOVER_STATE; // state of individual mover for extern to read
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

■ Software Design Example:

- Distributes only required methods and properties.
- Enables strict separation of concerns and connects classes horizontally
- I_McAxis is the base interface and is not optional.

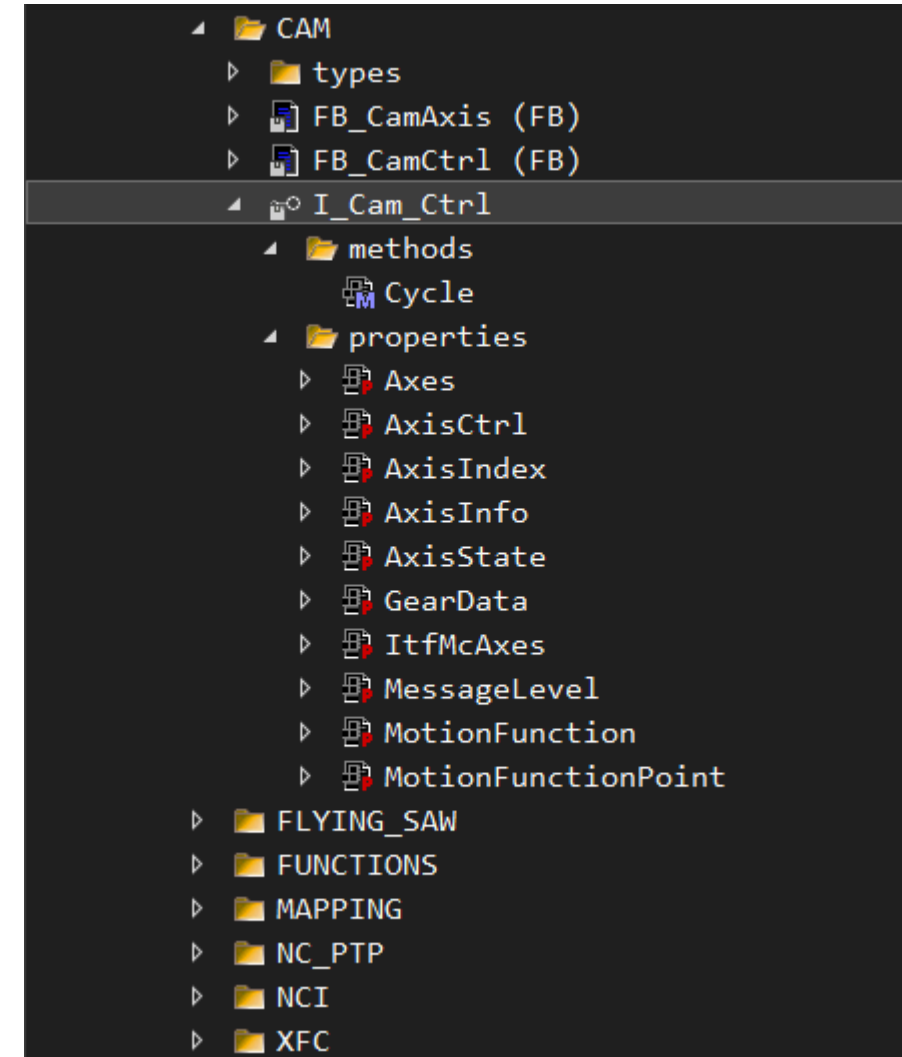
The Interface



■ Software Design Example:

The Interface

- Distributes only cycle method. The commands are written onto the facade structures.
- Enables use of empty interfaces for pre-compile defines
- Shall be void of any library datatypes from functions that are optional.



■ Software Design Example:

- The empty interface
- pre-compile defines
- Optional binding is possible with this
- The codebase does **not** need to change

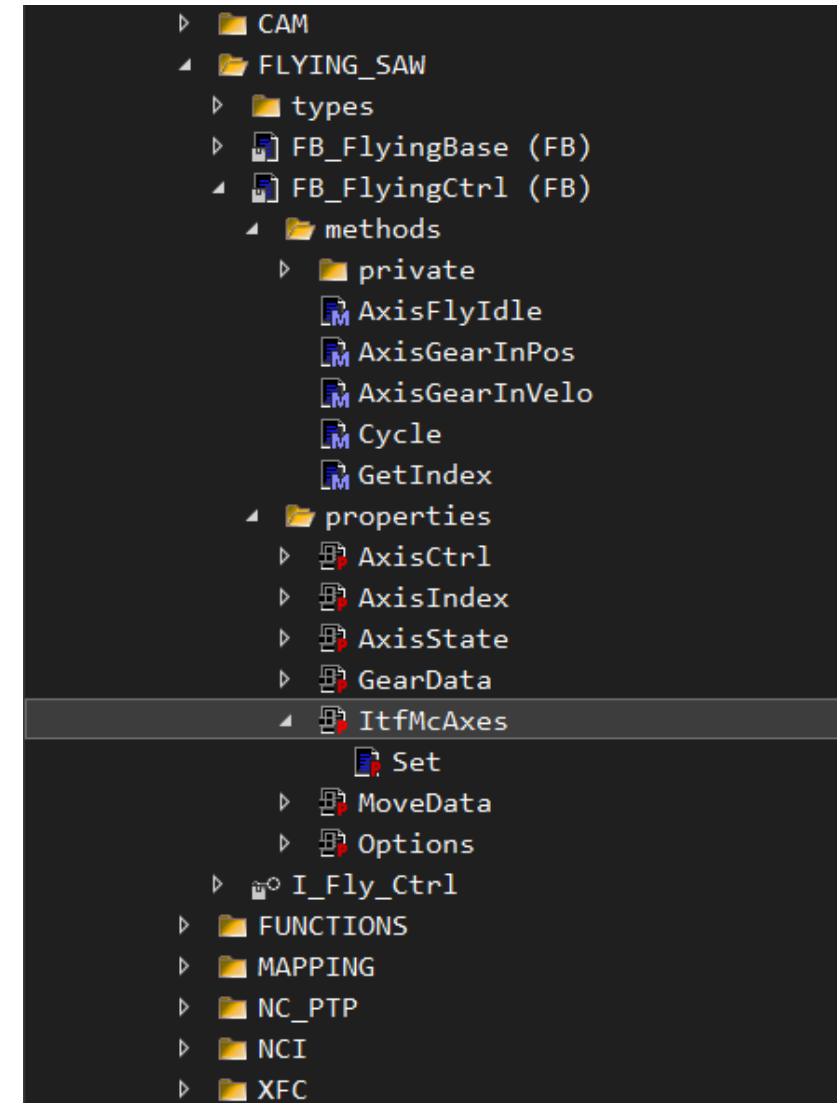
The Interface

```
//-----  
// cyclic interface function block: FB_CamCtrl  
// - MC_Camming functions are here  
//-----  
{IF defined (CAM)}  
  {info 'PLC_MOTION_LAYER: CAM'}  
  Control : ARRAY[1..PLC_CONSTANT.MAX_AXIS] OF FB_CamCtrl;  
{ELSE}  
  Control : ARRAY[1..PLC_CONSTANT.MAX_AXIS] OF I_Cam_Ctrl;  
{END_IF}  
END_VAR
```

■ Software Design Example:

- Injected in class via property
- Class is oblivious to implementation details
- This is the contract on offer

The Interface



- **Public on GitHub:**

- [https://github.com/haud-ba/PLC MOTION LAYER](https://github.com/haud-ba/PLC_MOTION_LAYER)
- [https://github.com/haud-ba/XTS TRANSPORT LAYER](https://github.com/haud-ba/XTS_TRANSPORT_LAYER)
- [https://github.com/HAUDX/XTS TRANSPORT EXAMPLES](https://github.com/HAUDX/XTS_TRANSPORT_EXAMPLES)

- **Reference Implementations:**

- **Schaeffler
Sondermaschinenbau (2026)** – **XTS_TRANSPORT_LAYER**
- **Transnova Ruf (2025)** – **XTS_TRANSPORT_LAYER**
- **CeramTech (2025)** – **XTS_TRANSPORT_LAYER (incl. NCT)**
- **Fischer
Sondermaschinenbau (since 2024)** – **PLC_MOTION_LAYER;
XTS_TRANSPORT_LAYER**
- **Karl Storz Medical (since 2024)** – **XTS_TRANSPORT_LAYER**
- **Mühlbauer AG (since 2020)** – **PLC_OPEN_DRIVE (exclusive, non-public)**